

研究生科研代码注意事项（完整版：版本管理、实验记录、可扩展性、自动化、异常处理与可视化总结）

这份文档面向研究生日常科研开发，目标不是“把代码写得很花”，而是让代码真正具备科研价值。一份合格的科研代码，至少要满足以下六个要求：

1. **能复现**：同样的代码和参数，过一周、一个月、半年后还能跑出同样结果。
2. **能追踪**：知道每个结果来自哪一版代码、哪一组参数、哪一次实验。
3. **能扩展**：今天能做一个实验，明天能很方便扩展成一组实验。
4. **能自动化**：批量实验、日志输出、结果汇总、画图分析尽量自动完成。
5. **能定位问题**：程序出错时能快速知道哪里错、为什么错。
6. **能总结结论**：实验结束后能用图表和交互方式快速看出规律，而不是手工翻日志。

很多论文难以复现、结果前后不一致、后续学生无法接手，往往不是算法本身的问题，而是代码管理、实验记录和自动化流程做得不够规范。

一、版本管理：多用 Git，命名规范，过程可追踪

版本管理的核心目的不是“备份代码”，而是：

- 记录每次改了什么
- 知道哪版代码对应哪组实验结果
- 出错时能快速回退
- 多人协作时避免互相覆盖
- 论文投稿和修改时快速定位结果来源

1.1 一定要使用 Git，不要手动保存多个最终版文件夹

错误示例：

```
final_code.py  
final_code2.py  
final_code_final.py  
final_code_final_new.py  
final_code_final_new2_真的最终版.py
```

这种做法最大的问题是：

- 不知道每一版改了什么
- 不知道哪个版本对应论文图 3
- 容易误删和覆盖
- 文件越来越乱
- 过几天自己都看不懂

正确做法是：

- 一个项目对应一个 Git 仓库
- 每完成一个明确修改就 commit
- 重要实验节点打 tag
- 新功能和新思路使用 branch

1.2 推荐项目目录结构

建议一个科研项目至少组织成下面这样：

```
project/
|
├─ configs/                # 配置文件
|   │─ default.json
|   │─ noise_sweep.json
|   │─ field_data.json
|   └─ ablation_hash.json
|
├─ data/                   # 数据或数据说明
|   │─ raw/
|   │─ processed/
|   └─ README.md
|
├─ scripts/               # Bash / 调度脚本
|   │─ run_train.sh
|   │─ run_batch.sh
|   │─ run_parallel.sh
|   └─ summarize_results.sh
|
├─ src/                   # 核心源码
|   │─ train.py
|   │─ test.py
|   │─ dataset.py
|   │─ models/
|   │─ losses.py
|   │─ trainer.py
|   │─ metrics.py
|   │─ utils/
|   └─ visualization/
|
├─ experiments/           # 每次实验独立输出目录
|   │─ exp_20260416_101500_snr10_seed42/
|   │─ exp_20260416_102300_snr12_seed43/
|   └─ summary/
|
├─ notebooks/            # 仅用于分析和临时探索
├─ tests/                # 单元测试或简单检查脚本
├─ README.md
├─ requirements.txt
├─ environment.yml
└─ .gitignore
```

优点：

- 代码、配置、数据、结果分离
- 目录清晰，便于接手
- 方便写自动化脚本
- 后期维护成本低

参考文档：

- [如何写train模板，里面的几个回答都看看](#)
- [Template Demo1](#)
- [Template Demo2](#)

1.3 命名要规范：文件名、变量名、实验名都要见名知意

不推荐的命名

```
a = np.zeros((9, 20))
tmp = model(x)
dn = data_norm
dr = result
```

问题：

- 看不出来含义
- 容易误解
- 后期自己都忘记

推荐的命名

```
num_noise_levels = 9
num_trials = 20
snr_results = np.zeros((num_noise_levels, num_trials))

predicted_wavefield = model(input_coordinates)
normalized_seismic_data = normalize(seismic_data)
reconstructed_section = reconstruction_result
```

文件命名建议

推荐：

- train_inr.py
- test_interpolation.py
- plot_snr_curve.py
- summarize_experiments.py
- visualize_dashboard.py

避免：

- test.py
- new.py
- aaa.py
- 临时代码.py

1.4 Git commit 信息必须有意义

错误示例：

```
git commit -m "update"  
git commit -m "fix"  
git commit -m "修改"
```

这种 commit 几乎没有任何价值。

推荐写法：

```
git commit -m "add configurable noise sweep for SNR experiments"  
git commit -m "fix receiver coordinate normalization bug"  
git commit -m "refactor training loop into Trainer class"  
git commit -m "save final config and command to output directory"  
git commit -m "add interactive experiment dashboard"
```

好的 commit 信息至少要说明：

- 做了什么
- 为什么做

- 是否会影响实验结果

提交信息：规范命名，“回头看”能看懂

- **提交格式：** <type>: <subject> （类型+简短描述），类型可选：
 - feat：新功能（如添加维度解耦模块）
 - fix：修复bug（如修正SNR计算错误）
 - docs：文档更新（如补充README参数说明）
 - refactor：代码重构（不改变功能，如封装函数）
- **正反示例：**
 - ✗ 坏提交： `git commit -m "改了点代码"`
 - ✓ 好提交： `git commit -m "feat: 添加五维地震波场重建的张量隐式网络模块"`

1.5 分支管理建议

最简单实用的策略：

- main：稳定可复现版本
- dev：日常开发
- feature/...：新功能
- exp/...：临时实验思路

例如：

```
feature/add-hash-encoding
feature/add-result-dashboard
exp/test-lora-adaptation
exp/try-lower-learning-rate
```

1.6 重要实验版本打 tag

例如：

```
git tag v1.0-submitted
git tag v1.1-camera-ready
git tag exp-best-synthetic
git tag exp-field-data-final
```

这样后续：

- 写论文
- 改稿
- 回复审稿人
- 复现实验

都会方便很多。

1.7 一定要写 .gitignore

- **核心作用**：忽略临时文件、依赖包、大体积实验结果，避免仓库臃肿。否则会把缓存、结果、模型权重和临时文件全提交上去。
- **科研场景示例**（.gitignore 文件内容）：

```
__pycache__/  
*.pyc  
*.pkl  
*.npy  
*.npz  
*.mat  
*.h5  
*.log  
*.ckpt  
*.pth  
*.pt  
experiments/  
wandb/  
.vscode/  
.ipynb_checkpoints/
```

二、实验记录：输出所有参数，保证结果可复现

实验记录的核心原则：

任何一张图、一个表、一个数值结论，都必须能追溯到具体代码版本、参数配置和运行环境。

如果做不到，实验基本上就不算可复现。

2.1 所有参数必须显式输出

错误示例：

```
lr = 1e-3
epochs = 3000
batch_size = 4096
rank = 16
hidden_dim = 128
```

问题：

- 参数散落在代码里
- 很难追踪
- 最优结果怎么来的说不清楚

正确做法：

- 所有参数集中到配置文件
- 程序运行时自动保存一份“最终生效参数”

2.2 推荐配置文件示例

例如 `configs/default.json`：

```
{
  "experiment_name": "default_inr",
  "dataset_name": "synthetic_5d",
  "snr": 10,
  "seed": 42,
  "epochs": 3000,
  "learning_rate": 0.001,
  "batch_size": 4096,
  "hidden_dim": 128,
  "num_layers": 4,
  "activation": "sine",
  "model_type": "inr",
  "save_root": "experiments",
  "num_trials": 20,
  "noise_levels": [5, 7, 9, 11, 13, 15]
}
```

2.3 建议每次实验至少输出以下内容

基本信息

- 实验名称
- 时间戳
- 代码版本号
- Git commit hash
- 配置文件路径
- 启动命令

模型信息

- 模型类型
- 网络层数
- 隐层宽度
- 激活函数
- 编码方式
- rank / TT-rank / TR-rank

训练信息

- 学习率
- batch size
- epoch 数
- 优化器
- scheduler
- loss 权重
- 随机种子

数据信息

- 数据集名称
- 缺失率
- 噪声水平
- 数据维度
- 归一化方式
- 数据预处理方法

输出结果

- train / val / test loss
- SNR / PSNR / SSIM / RMSE
- 每轮训练时间
- 总耗时
- 最优 epoch
- 最优模型路径

2.4 每次实验都要自动生成独立输出目录

建议：

```
experiments/  
└─ exp_20260416_153000_snr10_seed42/  
    ├── config_original.json  
    ├── config_used.json  
    ├── run_command.sh  
    ├── launcher.sh  
    ├── git_commit.txt  
    ├── env_info.txt  
    ├── train.log  
    ├── metrics.json  
    ├── checkpoint_best.pth  
    ├── checkpoint_last.pth  
    ├── curves/  
    └─ figures/
```

这样以后找结果非常方便。

2.5 config 与 Bash 命令必须同时留痕

除了保存原始配置文件 `config_original.json`，还必须保存：

- `config_used.json`：最终生效参数
- `run_command.sh`：本次实验启动命令
- `launcher.sh`：发起实验的 Bash 脚本副本

因为真正重要的不只是“原始配置长什么样”，而是：

- 命令行覆盖了哪些参数
- 最终到底用了什么参数
- 这次实验是如何被启动的

2.6 Python 中保存最终参数示例

```
import os
import json
import argparse
from datetime import datetime

def load_config(config_path):
    with open(config_path, "r", encoding="utf-8") as f:
        return json.load(f)

def save_json(data, path):
    with open(path, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=4, ensure_ascii=False)

parser = argparse.ArgumentParser()
parser.add_argument("--config", type=str, required=True)
parser.add_argument("--snr", type=int, default=None)
parser.add_argument("--seed", type=int, default=None)
parser.add_argument("--output_dir", type=str, default=None)
args = parser.parse_args()

config = load_config(args.config)
config["config_path"] = args.config

if args.snr is not None:
    config["snr"] = args.snr
if args.seed is not None:
    config["seed"] = args.seed

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
if args.output_dir is None:
    exp_name = f"exp_{timestamp}_snr{config['snr']}_seed{config['seed']}"
    output_dir = os.path.join(config["save_root"], exp_name)
else:
    output_dir = args.output_dir

os.makedirs(output_dir, exist_ok=True)
save_json(config, os.path.join(output_dir, "config_used.json"))
```

2.7 同时保存运行命令、Git 版本与环境信息

```
import os
import sys
import shlex
import platform
import subprocess

command_str = "python " + " ".join(shlex.quote(arg) for arg in sys.argv)
with open(os.path.join(output_dir, "run_command.sh"), "w", encoding="utf-8") as f:
    f.write("#!/bin/bash\n")
    f.write(command_str + "\n")

try:
    git_commit = subprocess.check_output(["git", "rev-parse", "HEAD"]).decode().strip()
except Exception:
    git_commit = "unknown"

with open(os.path.join(output_dir, "git_commit.txt"), "w", encoding="utf-8") as f:
    f.write(git_commit + "\n")

env_lines = [f"python_version: {platform.python_version()}",
             f"platform: {platform.platform()}"]

with open(os.path.join(output_dir, "env_info.txt"), "w", encoding="utf-8") as f:
    f.write("\n".join(env_lines) + "\n")
```

2.8 日志不要只打印到终端，一定要存到对应文档

错误做法：

- loss 只打印在屏幕上
- 终端关掉后什么都没了

推荐：

- 终端打印
- 文件日志同步保存

例如：

```
import logging
import os

logging.basicConfig(
    filename=os.path.join(output_dir, "train.log"),
    level=logging.INFO,
    format="%asctime)s - %(levelname)s - %(message)s"
)

logging.info("Training started")
logging.info(f"snr={config['snr']}, seed={config['seed']}")
```

三、可扩展性：少写魔法数字，多配置化，多封装函数

可扩展性的核心：

今天代码能跑一个实验，明天来了新的数据或者任务也能方便地扩展成十个、一百个实验，而不是每次都推倒重来。

3.1 不要写魔法数字

错误示例：

```
snr_list_noise = np.zeros((9, 20))
```

这里的 9 和 20 是典型魔法数字。别人看不出来是什么意思，自己以后也容易忘。

正确做法：

```
num_noise_levels = config["num_noise_levels"]
num_trials = config["num_trials"]
snr_list_noise = np.zeros((num_noise_levels, num_trials))
```

配置文件中写：

```
{
    "num_noise_levels": 9,
    "num_trials": 20
}
```

3.2 参数统一放到 JSON / YAML 配置文件

错误做法：

```
noise_levels = [5, 7, 9, 11, 13, 15]
num_trials = 20
hidden_dim = 128
num_layers = 4
lr = 1e-3
```

推荐：

```
{
    "noise_levels": [5, 7, 9, 11, 13, 15],
    "num_trials": 20,
    "hidden_dim": 128,
    "num_layers": 4,
    "learning_rate": 0.001
}
```

3.3 大段逻辑必须抽象成函数

错误写法：

```
for epoch in range(epochs):  
    # 读数据  
    # 前向传播  
    # 计算loss  
    # backward  
    # 更新参数  
    # 验证  
    # 保存模型  
    # 打印日志  
    # 画图
```

这会让主程序越来越长，最终难以维护。

推荐拆分：

```
def train_one_epoch(model, dataloader, optimizer, criterion, device):  
    ...  
  
def validate(model, dataloader, criterion, device):  
    ...  
  
def save_checkpoint(model, optimizer, epoch, save_path):  
    ...  
  
def log_metrics(metrics, output_dir):  
    ...
```

主流程就会更清晰：

```
for epoch in range(config["epochs"]):  
    train_loss = train_one_epoch(model, train_loader, optimizer, criterion, device)  
    val_loss = validate(model, val_loader, criterion, device)  
    log_metrics({"epoch": epoch, "train_loss": train_loss, "val_loss": val_loss}, output_dir)
```

3.4 一类功能放一个模块，不要全堆在一个脚本里

推荐拆分：

- `dataset.py` : 数据读入与预处理
- `model.py` : 模型结构
- `losses.py` : 损失函数
- `trainer.py` : 训练逻辑
- `metrics.py` : 评价指标
- `summary.py` : 汇总实验结果
- `dashboard.py` : 可视化交互界面
- `utils.py` : 通用工具函数

3.5 尽量减少重复代码

错误示例：

```
if mode == "train":  
    # 一套数据处理  
elif mode == "val":  
    # 基本重复一遍  
elif mode == "test":  
    # 再复制一遍
```

更好的做法：

```
def prepare_batch(batch, device):  
    ...  
    return inputs, targets
```

训练、验证、测试都调用它。

3.6 主程序入口建议使用 argparse

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--config", type=str, required=True)
parser.add_argument("--seed", type=int, default=42)
parser.add_argument("--output_dir", type=str, required=True)
args = parser.parse_args()
```

这样便于脚本化、自动化、批量运行。

四、自动化：批量实验、结果汇总、画图分析 尽量脚本化

自动化的核心思想：

不要让人去手工重复做本来可以由代码自动完成的事情。

建议自动化的内容包括：

- 批量实验启动
- 参数扫描
- 多随机种子重复试验
- 输出目录生成
- config 保存
- run command 保存
- 日志保存
- 指标汇总
- 曲线绘制
- 最终结论可视化

4.1 用 Bash 驱动批量实验，而不是手动改代码

错误流程：

1. 打开 train.py
2. 改 snr = 10
3. 运行
4. 改 snr = 12
5. 再运行
6. 改 seed = 43
7. 再运行

这种流程非常低效，而且容易出错。

推荐 Bash 脚本：

```
#!/bin/bash

CONFIG=configs/default.json
SAVE_ROOT=experiments
mkdir -p "$SAVE_ROOT"

for snr in {5..15..2}; do
    for seed in 42 43 44; do
        timestamp=$(date +"%Y%m%d_%H%M%S")
        exp_name="exp_${timestamp}_snr${snr}_seed${seed}"
        output_dir="${SAVE_ROOT}/${exp_name}"
        mkdir -p "$output_dir"

        cmd="python train.py --config ${CONFIG} --snr ${snr} --seed ${seed} --output_dir ${output_dir}"

        echo "#!/bin/bash" > "${output_dir}/run_command.sh"
        echo "$cmd" >> "${output_dir}/run_command.sh"
        cp "$CONFIG" "${output_dir}/config_original.json"
        cp "$0" "${output_dir}/launcher.sh"

        bash "${output_dir}/run_command.sh" 2>&1 | tee "${output_dir}/train.log"
    done
done
```

4.2 批量实验结束后自动汇总结果

很多学生的常见问题是：

- 每个实验目录里有一个 `metrics.json`
- 但没有统一汇总
- 最后只能一个一个点开看

正确做法是再写一个汇总脚本，例如 `summarize_experiments.py`，自动扫描所有实验目录并生成：

- `summary.csv`
- `summary.json`
- 统计图
- 交互界面输入文件

示例：

```

import os
import json
import pandas as pd

def collect_metrics(experiments_root):
    records = []

    for exp_name in os.listdir(experiments_root):
        exp_dir = os.path.join(experiments_root, exp_name)
        metrics_path = os.path.join(exp_dir, "metrics.json")
        config_path = os.path.join(exp_dir, "config_used.json")

        if os.path.exists(metrics_path) and os.path.exists(config_path):
            with open(metrics_path, "r", encoding="utf-8") as f:
                metrics = json.load(f)
            with open(config_path, "r", encoding="utf-8") as f:
                config = json.load(f)

            record = {}
            record.update(config)
            record.update(metrics)
            record["experiment_dir"] = exp_dir
            records.append(record)

    return pd.DataFrame(records)

df = collect_metrics("experiments")
df.to_csv("experiments/summary/summary.csv", index=False)

```

4.3 所有流程尽量形成一条自动化流水线

推荐形成如下流程：

1. 读取配置文件
2. 创建输出目录
3. 保存命令、配置、环境、版本信息
4. 训练模型
5. 保存日志和指标
6. 汇总所有实验结果

7. 自动画图
8. 自动生成交互式结论界面
9. 输出最终 summary 文件

也就是说，尽量做到：

从启动实验到看到总结结论，中间少依赖手工操作。

五、并行：多用并行，但必须有资源控制和隔离意识

在科研中，很多实验彼此独立，例如：

- 不同 SNR
- 不同随机种子
- 不同学习率
- 不同网络结构

这种实验非常适合并行。

但并行不是简单加一个 `&` 就完事了，而是要做到：

- 控制最大并发数
- 隔离输出目录
- 分离日志
- 合理使用 CPU / GPU / 内存

5.1 最简单的 Bash 并行方式：后台运行 `&`

```
bash run_command.sh > train.log 2>&1 &
```

优点：

- 简单
- 少量任务方便

缺点：

- 很容易一下子起太多进程
- 可能打满资源
- 不适合大规模实验管理

5.2 推荐设置最大并发数

例如同时最多跑 2 个：

```
MAX_JOBS=2
job_count=0

for snr in {5..15..2}; do
    for seed in 42 43 44; do
        run_job "$snr" "$seed" &
        ((job_count++))

        if (( job_count % MAX_JOBS == 0 )); then
            wait
        fi
    done
done

wait
```

5.3 GPU 并行必须显式分配设备

例如：

```
CUDA_VISIBLE_DEVICES=0 python train.py ...
CUDA_VISIBLE_DEVICES=1 python train.py ...
```

或者在脚本中轮换设备：

```
gpus=(0 1)
gpu_index=0

for snr in {5..15..2}; do
    for seed in 42 43 44; do
        gpu=${gpus[$gpu_index]}
        gpu_index=$(( (gpu_index + 1) % ${#gpus[@]} ))

        CUDA_VISIBLE_DEVICES=$gpu python train.py --snr $snr --seed $seed &
    done
done

wait
```

但必须考虑：

- 单个任务显存占用多少
- 一个 GPU 能承受几个任务
- 是否会 OOM

5.4 并行运行必须满足三条底线

1. 不同实验不能共用同一输出目录
2. 不同实验不能共用同一个日志文件
3. GPU 任务不能无控制抢占显存

参考资料：[tmux](#)

六、异常处理：所有流程都要考虑“如果失败怎么办”

很多科研代码的问题不是“永远不能跑”，而是：

- 数据路径写错就直接崩

- 配置文件少一个字段就莫名其妙报错
- 训练中途显存不足，结果目录里什么也没留下
- 某次实验失败后，整个批处理脚本直接停止

所以科研代码一定要有异常处理意识。

6.1 配置文件读取要做检查

错误写法：

```
with open(config_path, "r") as f:
    config = json.load(f)
```

更稳妥的写法：

```
import os
import json

if not os.path.exists(config_path):
    raise FileNotFoundError(f"Config file not found: {config_path}")

with open(config_path, "r", encoding="utf-8") as f:
    config = json.load(f)
```

6.2 关键字段缺失要尽早报错

```
required_keys = ["snr", "seed", "epochs", "learning_rate", "save_root"]

for key in required_keys:
    if key not in config:
        raise KeyError(f"Missing required config key: {key}")
```

6.3 数据加载要检查文件存在和维度合理性

```
if not os.path.exists(data_path):
    raise FileNotFoundError(f"Data file not found: {data_path}")

data = np.load(data_path)
if data.ndim != 5:
    raise ValueError(f"Expected 5D seismic data, but got shape {data.shape}")
```

6.4 批量实验脚本要允许单个实验失败而不影响全局

例如在 Bash 中，可以写：

```
if bash "${output_dir}/run_command.sh" > "${output_dir}/train.log" 2>&1; then
    echo "SUCCESS" > "${output_dir}/status.txt"
else
    echo "FAILED" > "${output_dir}/status.txt"
    echo "Experiment failed: ${output_dir}"
fi
```

这样即使某个实验失败了，其他实验还能继续。

6.5 Python 中建议用 try-except 捕获高风险步骤

例如：

```
try:
    train_loss = train_one_epoch(model, train_loader, optimizer, criterion, device)
except RuntimeError as e:
    logging.exception("Training failed due to runtime error")
    raise e
```

或者在外层统一处理：

```
try:
    main()
except Exception as e:
    logging.exception("Experiment failed")
    raise
```

这样日志里至少能保留堆栈信息，便于定位问题。

七、多用 assert：尽早发现错误，防止错误结果悄悄传播

assert 的作用不是替代异常处理，而是：

在程序假定某些条件必然成立时，尽早检查这些条件。

如果条件不成立，就立即终止，而不是让错误继续传递下去。

7.1 参数范围检查

```
assert config["epochs"] > 0, "epochs must be positive"
assert config["learning_rate"] > 0, "learning rate must be positive"
assert config["batch_size"] > 0, "batch size must be positive"
assert config["snr"] >= 0, "snr should be non-negative"
```

7.2 数据维度检查

```
assert data.ndim == 5, f"Expected 5D data, got {data.ndim}D"
assert data.shape[0] > 0, "First dimension must be non-empty"
```

7.3 模型输入输出形状检查

```
prediction = model(inputs)
assert prediction.shape == targets.shape, \
    f"Prediction shape {prediction.shape} does not match target shape {targets.shape}"
```

7.4 训练过程中的数值检查

```
loss = criterion(prediction, targets)
assert torch.isfinite(loss).all(), "Loss contains NaN or Inf"
```

也可以检查梯度：

```
for name, param in model.named_parameters():
    if param.grad is not None:
        assert torch.isfinite(param.grad).all(), f"Gradient has NaN or Inf in {name}"
```

7.5 汇总结果时检查关键文件存在

```
assert os.path.exists(metrics_path), f"Missing metrics file: {metrics_path}"
assert os.path.exists(config_path), f"Missing config file: {config_path}"
```

7.6 使用 assert 的注意事项

适合 assert 的场景：

- 程序内部假设检查
- 开发调试阶段的快速定位
- 输入输出形状一致性检查
- 数值有效性检查

不适合只用 `assert` 的场景：

- 用户输入错误
- 文件不存在
- 外部资源异常
- 网络 / 磁盘 / 显存不足

这些情况仍然应该用明确异常处理。

八、可视化交互：方便总结实验结论，而不是手工翻日志

这是很多学生最容易忽略、但实际上非常重要的一点。

科研代码的目标不只是“输出很多实验目录”，而是：

实验做完后，能够高效总结规律，支持决策和写论文。

如果没有一个好的可视化总结流程，最后就会出现：

- 一堆 `metrics.json`
- 一堆 `train.log`
- 一堆 `png`
- 但很难快速看出哪组参数最好、哪种趋势最稳定、哪种方法最值得写进论文

所以建议增加一个**交互式实验总结界面**。

8.1 交互式总结界面应该解决什么问题

建议至少支持下面几个功能：

1. 浏览所有实验结果
2. 按参数筛选实验，例如只看某个 SNR 或某个模型
3. 按指标排序，例如按 SNR、PSNR、RMSE 排序
4. 对比不同随机种子结果波动
5. 画趋势图，例如 SNR vs. noise level

6. 自动计算均值和标准差
7. 快速定位最佳实验目录
8. 点击后查看对应 config、日志、图像和权重路径

8.2 推荐使用方式

可以分为两层：

第一层：自动生成静态图

例如：

- loss 曲线
- 不同参数下的指标折线图
- 条形图
- 箱线图
- heatmap

第二层：交互式界面

例如使用：

- Streamlit
- Gradio
- Dash
- Plotly

其中对研究生最友好的通常是 **Streamlit**，因为简单直接。

8.3 一个推荐的交互式总结界面结构

例如 `dashboard.py`：

```

import os
import pandas as pd
import streamlit as st

summary_path = "experiments/summary/summary.csv"
df = pd.read_csv(summary_path)

st.title("Experiment Summary Dashboard")

selected_model = st.selectbox("Model Type", sorted(df["model_type"].dropna().unique()))
selected_snr = st.multiselect("SNR", sorted(df["snr"].dropna().unique()), default=sorted(df["snr"]

filtered_df = df[(df["model_type"] == selected_model) & (df["snr"].isin(selected_snr))]

st.write("Filtered Results")
st.dataframe(filtered_df)

st.line_chart(filtered_df.groupby("snr")["test_snr"].mean())
st.bar_chart(filtered_df.groupby("seed")["test_snr"].mean())

```

运行：

```
streamlit run dashboard.py
```

这类界面非常适合：

- 组会汇报前快速看结果
- 总结不同参数趋势
- 发现异常实验
- 写论文时筛选图表

8.4 建议自动生成 summary.csv 作为交互输入

整个交互界面不应该直接去逐个读原始实验目录，而应该基于统一的 `summary.csv`。

推荐流程：

1. 每个实验输出自己的 `config_used.json` 和 `metrics.json`
2. `summarize_experiments.py` 扫描全部实验目录

3. 自动生成 `summary.csv`

4. `dashboard.py` 直接读取 `summary.csv`

这样好处是：

- 结构更清晰
- 运行更快
- 可扩展性更好
- 方便分享和归档

8.5 建议加入“实验结论自动总结”的辅助模块

除了图表和表格，也可以自动生成一些文本统计信息，例如：

- 最优实验是哪一个
- 平均性能最好的是哪种模型
- 哪种参数对结果最敏感
- 随机种子波动是否明显
- 某些实验是否异常偏离

例如：

```
best_row = df.loc[df["test_snr"].idxmax()]
print("Best experiment:")
print(best_row[["experiment_dir", "snr", "seed", "test_snr"]])
```

进一步还可以自动输出一段总结：

在当前实验中，`SNR=11`、`seed=43` 的模型表现最好，`test_snr` 达到 12.53 dB。

从整体趋势来看，随着噪声水平增加，重建性能逐渐下降，但在 `SNR=9` 到 `11` 区间模型表现相对稳定。不同随机种子之间存在一定波动，建议最终报告结果时同时汇报均值与标准差。

这种自动总结对写周报、组会和论文都很有帮助。

九、推荐的完整自动化流程

建议大家逐步把项目组织成下面这条完整流水线：

9.1 单次实验流程

1. 读取 config
2. 检查参数合法性
3. 创建输出目录
4. 保存原始 config、最终 config、运行命令、代码版本、环境信息
5. 开始训练
6. 持续写日志
7. 保存 checkpoint
8. 保存 metrics
9. 保存曲线和图像
10. 标记成功或失败状态

9.2 批量实验流程

1. Bash 扫描参数组合
2. 为每个实验生成独立输出目录
3. 支持并行或限速并行
4. 每个实验失败不影响整体流程
5. 批量运行结束后自动汇总全部结果
6. 自动生成 summary.csv 和静态图
7. 启动交互式 dashboard 查看规律
8. 自动辅助总结最优结论与趋势结论

十、推荐直接遵守的“实验室规范”

下面这段可以直接发给学生：

1. 所有项目必须使用 Git 管理，禁止用多个“最终版”文件夹管理版本。

2. 所有参数必须配置化，统一写入 JSON/YAML，禁止散落在代码各处。
3. 每次实验必须自动保存原始配置、最终配置、运行命令、代码版本、环境信息、日志和指标。
4. 每次实验必须使用独立输出目录，禁止不同实验共用目录。
5. 批量实验尽量使用 Bash 或调度脚本自动运行，禁止手工反复改源码。
6. 多个独立实验应尽量采用并行方式提高效率，但必须控制最大并发数并合理分配 GPU。
7. 程序关键位置必须增加异常处理和 assert 检查，尽早发现问题。
8. 训练结束后必须自动生成统一 summary 文件，用于后续统计、作图和结论总结。
9. 建议增加交互式结果界面，用于筛选实验、比较参数和快速总结规律。
10. 论文中的每张图、每个表和每个结论，都必须能追溯到具体实验目录、配置文件和代码版本。

十一、一句话总结

科研代码最重要的不是“先跑出来”，而是做到：

参数留痕、命令留痕、版本留痕、日志留痕、异常可查、流程自动、结论可视、结果可复现。

只有做到这些，代码才真正具备科研积累价值，而不只是一次性的“能跑脚本”。